EXPERIMENT NO. 4

FREQUENCY DOMAIN IMAGE FILTERING USING FOURIER TRANSFORM

In [1]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. LOAD GRAYSCALE IMAGE
image_path = input("Enter the path of grayscale image: ")
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: Image could not be loaded!")
    exit()

print("Image loaded successfully!")
print("Image Shape:", image.shape)


# 2. DISPLAY ORIGINAL IMAGE
plt.imshow(image, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()


# 3. COMPUTE DISCRETE FOURIER TRANSFORM (DFT)
dft = cv2.dft(np.float32(image), flags=cv2.DFT_COMPLEX_OUTPUT)
print("DFT calculated successfully.")


# 4. SHIFT ZERO FREQUENCY TO CENTER
dft_shift = np.fft.fftshift(dft)
print("Frequency spectrum shifted to center.")


# 5. CALCULATE MAGNITUDE SPECTRUM
magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
magnitude_spectrum = 20 * np.log(magnitude + 1)


# 6. DISPLAY FREQUENCY SPECTRUM
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")
plt.show()


# 7. CREATE LOW-PASS FILTER
rows, cols = image.shape
crow, ccol = rows // 2, cols // 2
radius = 50
```

```
low_pass_mask = np.zeros((rows, cols, 2), np.uint8)
cv2.circle(low_pass_mask, (ccol, crow), radius, (1, 1), -1)

print("Low-Pass Filter created.")


# 8. APPLY LOW-PASS FILTER
low_pass_dft = dft_shift * low_pass_mask


# 9. INVERSE FOURIER TRANSFORM FOR LOW-PASS
low_shift = np.fft.ifftshift(low_pass_dft)
low_image = cv2.idft(low_shift)

low_image = cv2.magnitude(
    low_image[:, :, 0],
    low_image[:, :, 1]
)

low_image = cv2.normalize(
    low_image, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)


# 10. DISPLAY LOW-PASS FILTERED IMAGE
plt.imshow(low_image, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")
plt.show()


# 11. CREATE HIGH-PASS FILTER
high_pass_mask = np.ones((rows, cols, 2), np.uint8)
cv2.circle(high_pass_mask, (ccol, crow), radius, (0, 0), -1)

print("High-Pass Filter created.")


# 12. APPLY HIGH-PASS FILTER
high_pass_dft = dft_shift * high_pass_mask


# 13. INVERSE FOURIER TRANSFORM FOR HIGH-PASS
high_shift = np.fft.ifftshift(high_pass_dft)
high_image = cv2.idft(high_shift)

high_image = cv2.magnitude(
    high_image[:, :, 0],
    high_image[:, :, 1]
)

high_image = cv2.normalize(
    high_image, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)


# 14. DISPLAY HIGH-PASS FILTERED IMAGE
plt.imshow(high_image, cmap="gray")
plt.title("High-Pass Filtered Image")
```

```
plt.axis("off")
plt.show()


# 15. DISPLAY ALL RESULTS TOGETHER
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(low_image, cmap="gray")
plt.title("Low-Pass Filter")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(high_image, cmap="gray")
plt.title("High-Pass Filter")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(low_pass_mask[:, :, 0], cmap="gray")
plt.title("Low-Pass Mask")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(high_pass_mask[:, :, 0], cmap="gray")
plt.title("High-Pass Mask")
plt.axis("off")

plt.tight_layout()
plt.show()


# 16. SAVE OUTPUT IMAGES
cv2.imwrite("original_image.jpg", image)

cv2.imwrite(
    "magnitude_spectrum.jpg",
    cv2.normalize(
        magnitude_spectrum, None, 0, 255, cv2.NORM_MINMAX
    ).astype(np.uint8)
)

cv2.imwrite("low_pass_filtered.jpg", low_image)
cv2.imwrite("high_pass_filtered.jpg", high_image)


# 17. FINAL OUTPUT
print("\n" + "=" * 50)
print("EXPERIMENT NO. 4 COMPLETED SUCCESSFULLY")
print("=" * 50)
```

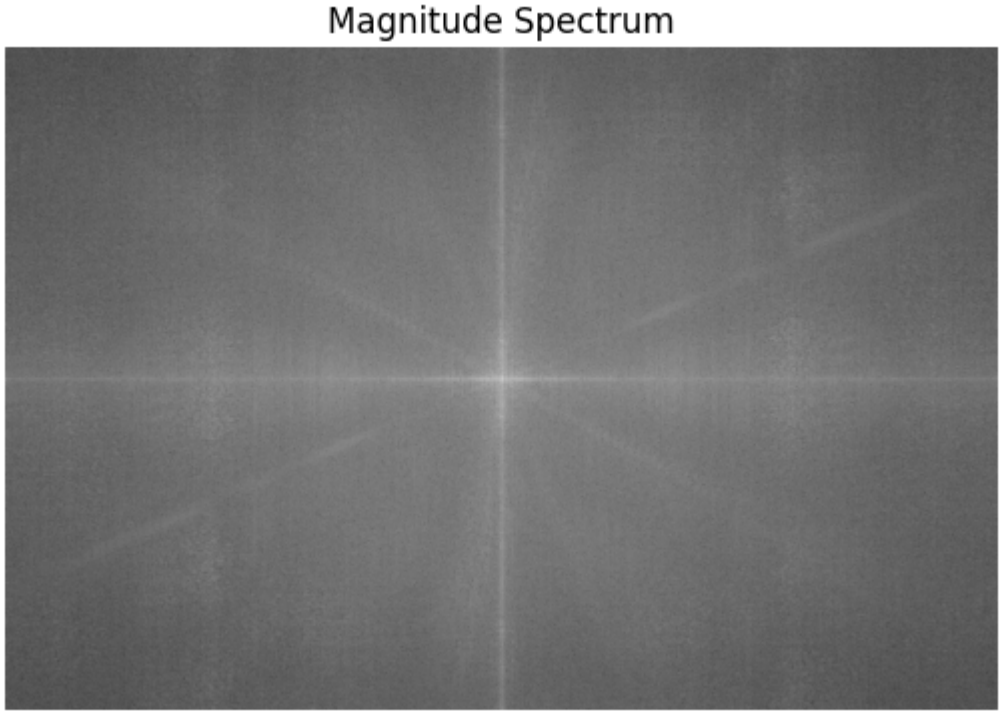
```
print("✓ Grayscale Image Loaded")
print("✓ DFT Calculated")
print("✓ Frequency Spectrum Created")
print("✓ Magnitude Spectrum Calculated")
print("✓ Low-Pass Filtering Applied")
print("✓ High-Pass Filtering Applied")
print("✓ IDFT Applied")
print("✓ Output Images Saved")

print("=" * 50)
```

```
Image loaded successfully!
Image Shape: (1024, 1536)
```

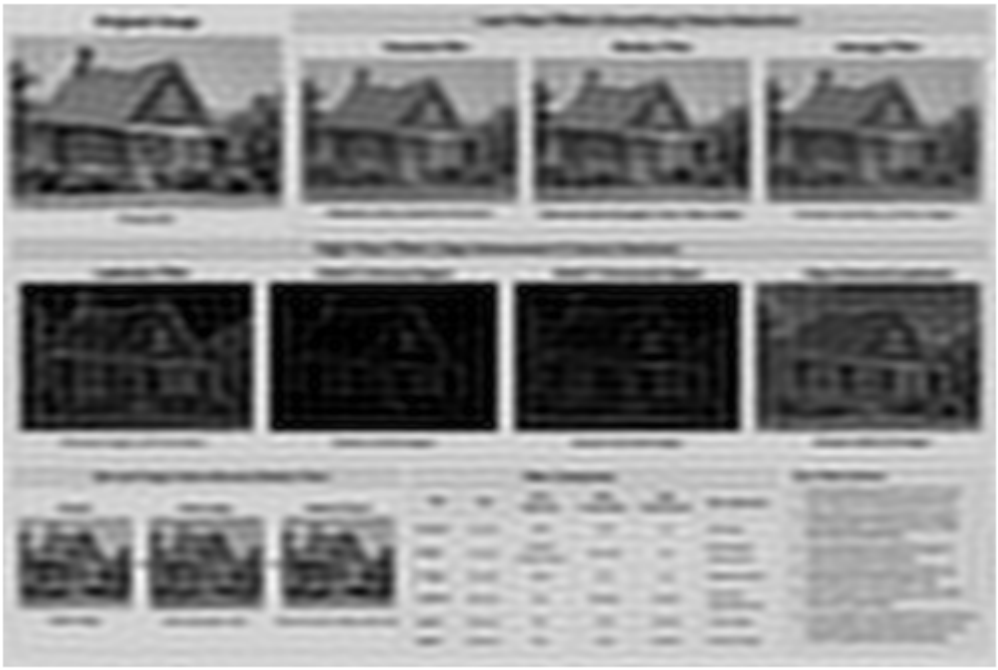



```
DFT calculated successfully.
Frequency spectrum shifted to center.
```




```
Low-Pass Filter created.
```

Low-Pass Filtered Image



```
High-Pass Filter created.
```

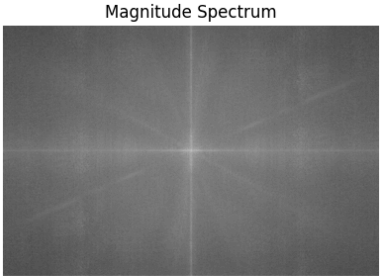
High-Pass Filtered Image



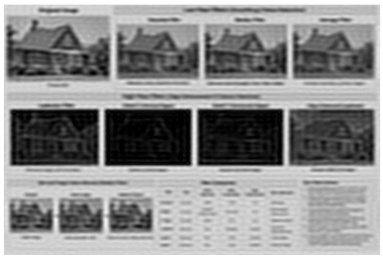
Original Image



Magnitude Spectrum



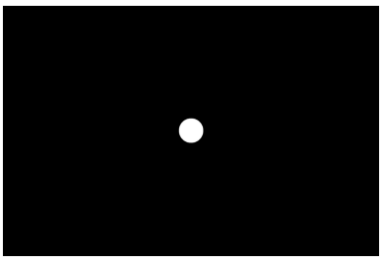
Low-Pass Filter



High-Pass Filter



Low-Pass Mask



High-Pass Mask



```
==================================================
EXPERIMENT NO. 4 COMPLETED SUCCESSFULLY
==================================================
✓ Grayscale Image Loaded
✓ DFT Calculated
✓ Frequency Spectrum Created
✓ Magnitude Spectrum Calculated
✓ Low-Pass Filtering Applied
✓ High-Pass Filtering Applied
✓ IDFT Applied
✓ Output Images Saved
==================================================
```

Question

1. What is the Fourier Transform? Why is it important in digital image processing?

The **Fourier Transform (FT)** converts an image from the **spatial domain to the frequency domain**.

It is important because:

- It analyzes frequency components.
- It allows frequency-based filtering.
- Low frequencies represent smooth regions.
- High frequencies represent edges, details, and noise.
- It is useful for image enhancement and restoration.

---

2. Differentiate between the spatial domain and the frequency domain.

Spatial Domain	Frequency Domain
Works directly on pixels.	Works on frequency components.
Uses pixel intensity values.	Uses low and high frequencies.

Spatial Domain	Frequency Domain
Uses kernels/masks.	Uses frequency filters.
Examples: Median, Gaussian filters.	Examples: Low-Pass, High-Pass filters.

---

3. What is the significance of the Discrete Fourier Transform (DFT) in image processing?

The **DFT** converts a digital image into its frequency components.

It is useful for:

- Frequency analysis.
- Image filtering.
- Noise reduction.
- Image enhancement.
- Image restoration.

Example:

```
dft = cv2.dft(
    np.float32(image),
    flags=cv2.DFT_COMPLEX_OUTPUT
)
```

---

4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

`fftshift()` moves the **zero-frequency component from the corner to the center**.

`dft_shift = np.fft.fftshift(dft)`
After shifting:

- Low frequencies appear near the center.
- High frequencies appear farther away.
- The spectrum becomes easier to visualize.
- Frequency filters become easier to design.

---

5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

Low-Pass Filter	High-Pass Filter
Allows low frequencies.	Allows high frequencies.
Suppresses high frequencies.	Suppresses low frequencies.
Produces smoother images.	Enhances edges and details.
Reduces high-frequency noise.	Can emphasize edges and noise.
Used for smoothing.	Used for edge enhancement.

Applications:

- **LPF:** Noise reduction and image smoothing.
- **HPF:** Edge detection and detail enhancement.

---

6. What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?

The **IDFT** converts the filtered frequency-domain image back into the **spatial domain**.

```
result = cv2.idft(filtered_dft)
```

Process:

```
Image → DFT → Filtering → IDFT → Filtered Image
```

Thus, IDFT is used to reconstruct the final image.

---

7. Why is frequency domain filtering preferred for certain image enhancement tasks?

Frequency domain filtering allows selective manipulation of frequency components.

It helps to:

- Reduce noise.
- Enhance edges.
- Remove unwanted frequencies.
- Improve image quality.
- Analyze frequency characteristics.

---

8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

Four applications are:

1. Medical Image Enhancement Used for processing and enhancing medical images.

2. Satellite Image Analysis Used to analyze frequency patterns in remote sensing images.

3. Image Restoration Used to reduce noise and restore degraded images.

4. Biometric Systems Used for processing biometric image patterns.

Other applications include:

- Image compression.
- Texture analysis.
- Pattern recognition.
- Edge enhancement.

---

9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Spatial Domain Filtering	Frequency Domain Filtering
Works directly on pixels.	Works on frequency components.
Uses kernels/masks.	Uses frequency masks.
Simple for local operations.	Useful for frequency-selective operations.
Examples: Median, Gaussian.	Examples: LPF, HPF.
Often efficient for small kernels.	Useful for large/complex filtering.
Used for smoothing and noise removal.	Used for enhancement and restoration.

---

10. How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?

Frequency filtering allows important and unwanted frequency components to be separated.

It helps to:

- Reduce noise.
- Improve image quality.
- Enhance edges.
- Preserve useful information.
- Highlight fine details.
- Prepare images for feature extraction.

Process:

```
Input Image
     ↓
Fourier Transform
     ↓
Frequency Filtering
     ↓
Inverse Fourier Transform
     ↓
Enhanced / Restored Image
     ↓
Feature Extraction
```

Thus, frequency domain filtering is useful for **restoration, enhancement, noise removal, and feature extraction**.